

AI-Powered SoC Reference Manual Generation Platform

Technical Project Report

NXP Semiconductors — Documentation Engineering

Author: Jayant Kaushik · Principal Technical Writer

June 2025

This document is a comprehensive technical description of the AI-powered SoC Reference Manual Generation Platform designed and built at NXP Semiconductors. It covers motivation, system architecture, file format specifications, implementation patterns, code samples, and measurable outcomes. Proprietary file types are described using intuitive, purpose-derived names.

1. Executive Summary

Large-scale SoC Reference Manual (RM) authoring at semiconductor companies is among the most documentation-intensive and error-prone activities in chip development. A typical SoC integrates dozens to hundreds of distinct IP blocks, each with hundreds of individually parameterizable registers. The resulting reference manual may span tens of thousands of pages and must remain precisely synchronized with a constantly evolving RTL design database across multiple tape-out cycles.

At NXP Semiconductors, this process was largely manual. Technical writers and IP architects manually extracted parameter values from design databases, hand-crafted XML content structures, authored repetitive SoC-level chapters from scratch, managed hundreds of cross-references between spec files and design source, and maintained parallel spreadsheets of document-specific parameters not present in the RTL. This approach introduced significant inconsistency, consumed weeks of engineering time per tape-out, and created a chronic source of post-release errata.

This project — an AI-Powered Reference Manual Generation Platform — replaces that workflow with a largely automated pipeline delivered as a VSCode plugin and supporting toolchain. The platform integrates Verific-based RTL analysis and static elaboration, a hierarchical content architecture for IP/subsystem/SoC spec composition, automated spec source generation, a JSON-based non-RTL parameter database, Claude-powered SoC chapter authoring, DITA-OT parameter-resolved publishing, RTL vs. spec register validation, and a browser-based RM Explorer with a RAG-powered natural language chatbot.

1.1 Key Outcomes

- Reference Manual generation time reduced from 3–4 weeks per tape-out cycle to under one working day for full SoC spec assembly
- 100% automated RTL parameter extraction via Verific elaboration — no manual design database queries
- Non-RTL parameter management moved from unstructured spreadsheets to a versioned, schema-governed JSON database
- Register-level cross-validation between RTL design source and spec files — automated elimination of a class of post-tape-out errata
- AI-authored SoC architecture chapters — reduced repetitive writing effort for standard section types
- Natural language RM querying answering in under 3 seconds, replacing 15–30 minute PDF navigation sessions

2. Background and Motivation

2.1 The SoC Documentation Challenge

A modern NXP SoC may integrate 80–120 distinct IP blocks — FlexCAN, FlexIO, LPSPI, LPUART, SAI, ENET, USB, PCIe, DDR controllers, security subsystems, and many others. Each IP block is parameterized: it may appear at multiple base addresses, with different numbers of instances, with feature flags enabled or disabled based on chip tier, and with varying register counts based on configuration.

Each IP has a reusable documentation specification in a standard directory structure. That spec must be published both standalone (for IP-level review) and as part of one or more SoC-level reference manuals. At the SoC level, the single-source content must be resolved with chip-specific parameter values — base addresses, instance counts, feature flags — that differ between product variants.

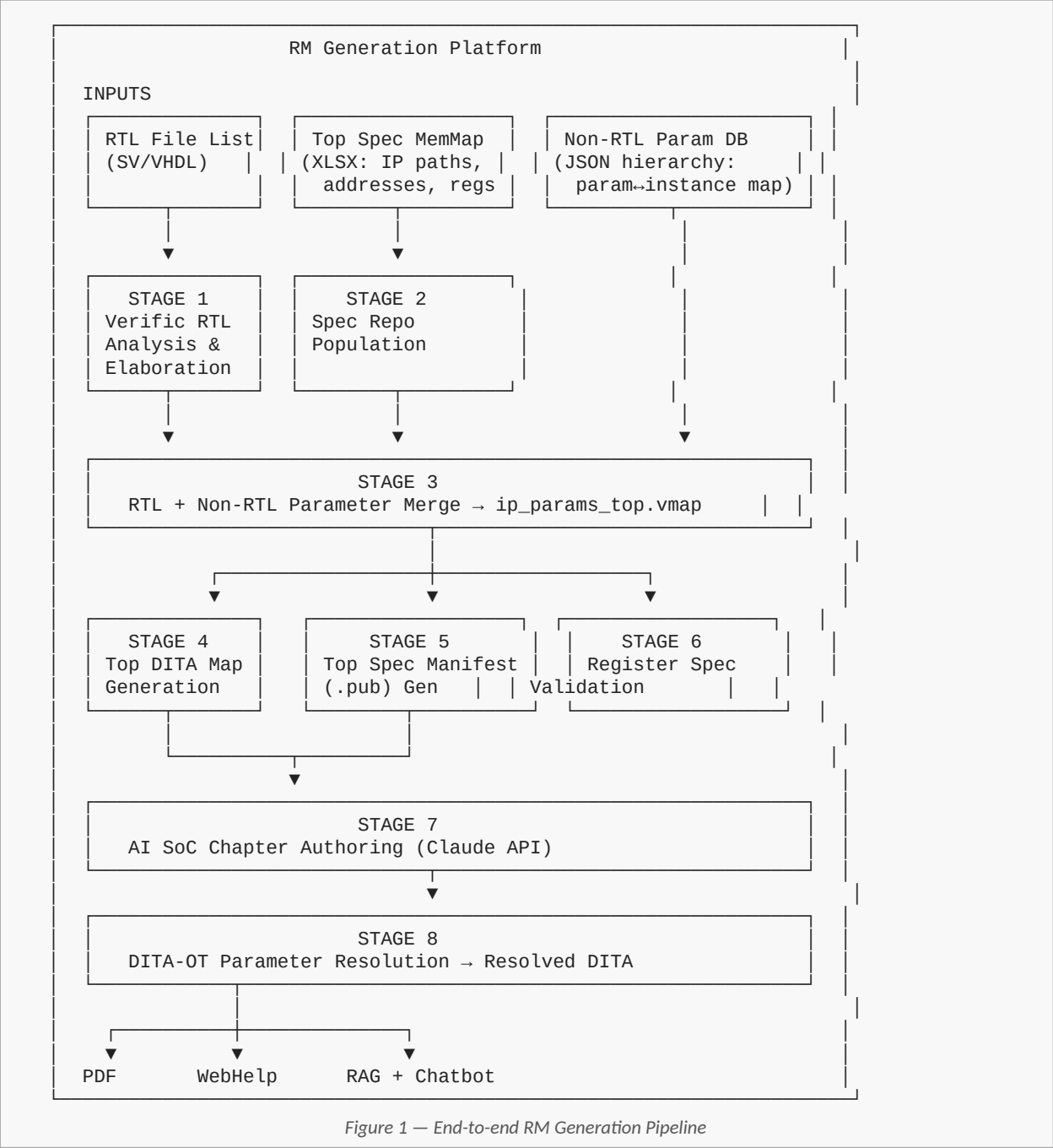
2.2 Limitations of the Legacy Workflow

Legacy Pain Point	Impact
Manual parameter extraction	Writers manually queried RTL databases or chased design engineers for current parameter values — slow, error-prone, version-sensitive
LEC-based extraction	Prior automated approach used logic equivalence checking tools for parameter extraction — inaccurate for this purpose, slow, hard to maintain
Spreadsheet parameter DB	Non-RTL parameters were tracked in per-chip spreadsheets with no schema enforcement, versioning, or traceability
Manual spec assembly	Top-level DITA maps, instance binding files, and memory map references were assembled manually — time-consuming and inconsistent
No register validation	Discrepancies between RTL register definitions and spec register files were not detected until post-silicon review
Static PDF navigation	Engineers navigating large RMs had no search or query capability beyond PDF text search

3. System Architecture

3.1 Pipeline Overview

The platform is structured as a sequential pipeline of eight stages, each producing artifacts consumed by downstream stages. The VSCode plugin exposes all stages through a command palette and sidebar panel. A local Python RPC server handles heavy computation, communicating with the plugin over JSON-RPC.



3.2 Spec Content Hierarchy

The platform operates on a three-tier content hierarchy. Each tier is a self-contained spec that can be published independently or assembled into a higher-tier spec.



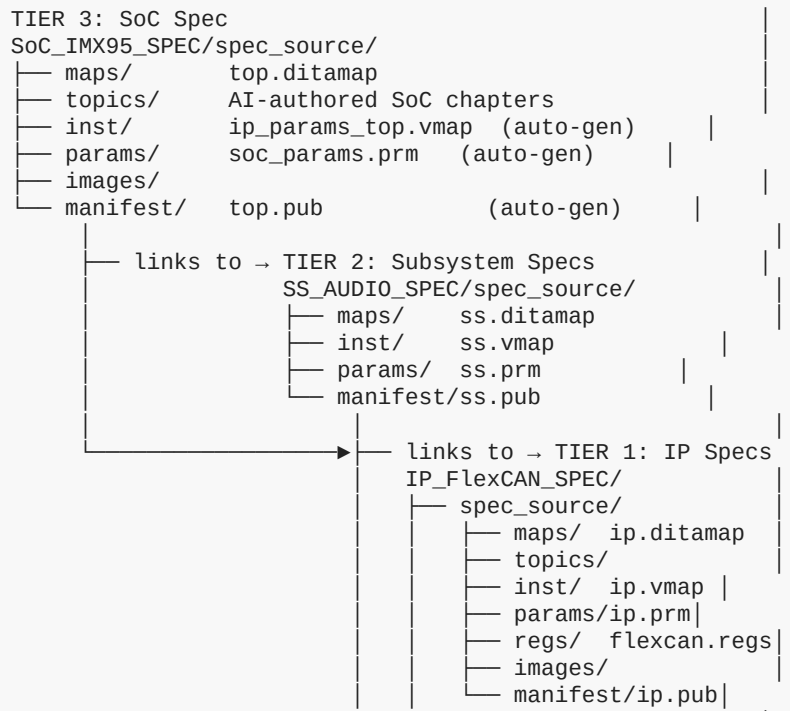


Figure 2 — Three-Tier Spec Content Hierarchy

4. Custom File Format Specifications

The platform introduces four custom file types that extend the standard DITA authoring model. These are proprietary to the NXP documentation engineering workflow but follow XML or JSON conventions. Each is described below with schema, semantics, and examples.

4.1 Spec Parameter Definition File (.prm)

The .prm file declares all parameters used within a spec tier. It acts as the schema authority — defining parameter name, data type, default value, allowed range or accepted values, and whether the parameter originates from the RTL database or is documentation-specific (non-RTL). A global .prm file defines publishing-level parameters shared across all specs.

Parameter Types

Type	Description
boolean	Accepts true or false only. Used for feature flags and enable/disable states.
integer	Any integer value. Used for instance counts, register widths, buffer sizes.
float	Any decimal value. Used for clock frequencies, timing parameters.
string	Free-form text. Used for instance names, revision identifiers, document metadata.
enumerated	A defined set of accepted string values. One value is instantiated per spec instance. Used for IP variants, bus widths, protocol modes.

```
<!-- ip.prm — IP-level parameter definition file (FlexCAN example) -->
<?xml version='1.0' encoding='UTF-8'?>
<prm id='flexcan-params' spec='IP_FlexCAN_SPEC'>
  <global-ref href='../global/publishing.prm' />

  <!-- RTL-sourced parameters -->
  <param name='NUM_MB' type='integer' source='rtl'
    default='64' low-limit='8' high-limit='128'
    description='Number of message buffers in this FlexCAN instance' />

  <param name='CLOCK_DOMAIN' type='enumerated' source='rtl'
    default='BUS_CLK'
    description='Clock domain assignment for this FlexCAN instance'>
    <accepted-value key='BUS_CLK' label='Bus clock domain' />
    <accepted-value key='CAN_CLK' label='CAN-specific clock domain' />
    <accepted-value key='ASYNC_CLK' label='Asynchronous clock domain' />
  </param>

  <!-- Non-RTL documentation parameters -->
  <param name='FEATURE_FDCAN' type='boolean' source='non-rtl'
    default='false'
    description='Whether FD-CAN is enabled for this chip configuration' />

  <param name='IP_REVISION' type='string' source='non-rtl'
    default='3.0'
    description='IP revision identifier for this chip tape-out' />

  <param name='BASE_ADDRESS' type='string' source='rtl'
    default='0x00000000'
    description='Base address of this IP instance in the SoC memory map' />
</prm>
```

4.2 Content Instance File (.vmap)

The .vmap file instantiates concrete parameter values for a spec, binding resolved values to the parameters declared in the corresponding .prm file. It also contains one or more Register Instance elements, each linking to a specific register description file (.regs or IPXACT) and carrying parameter value bindings that govern resolution and rendering of that register set. If the same register description file is linked by multiple Register Instance elements, each can carry different parameter bindings — enabling per-instance customization of shared register sets.

```
<!-- ip_params_top.vmap - merged RTL + non-RTL content instance file -->
<!-- This file is auto-generated by the RM Generation Platform -->
<?xml version='1.0' encoding='UTF-8'?>
<vmap id='imx95-flexcan-inst'
      spec='IP_FlexCAN_SPEC'
      chip='IMX95'
      params-ref='ip.prm'>

  <!-- Chip-level parameter overrides (RTL-sourced) -->
  <param-override name='NUM_MB' value='96' source='rtl'/>
  <param-override name='CLOCK_DOMAIN' value='CAN_CLK' source='rtl'/>
  <param-override name='BASE_ADDRESS' value='0x4A510000' source='rtl'/>

  <!-- Non-RTL overrides (documentation-specific) -->
  <param-override name='FEATURE_FDCAN' value='true' source='non-rtl'/>
  <param-override name='IP_REVISION' value='3.2' source='non-rtl'/>

  <!-- Register Instance: FLEXCAN_0 -->
  <reg-instance id='FLEXCAN_0'
               regs-ref='../regs/flexcan.regs'
               base-address='0x4A510000'
               instance-name='FLEXCAN_0'>
    <param-binding name='NUM_MB' value='96'/>
    <param-binding name='CLOCK_DOMAIN' value='CAN_CLK'/>
    <param-binding name='FEATURE_FDCAN' value='true'/>
  </reg-instance>

  <!-- Register Instance: FLEXCAN_1 - same regs file, different bindings -->
  <reg-instance id='FLEXCAN_1'
               regs-ref='../regs/flexcan.regs'
               base-address='0x4A520000'
               instance-name='FLEXCAN_1'>
    <param-binding name='NUM_MB' value='64'/>
    <param-binding name='CLOCK_DOMAIN' value='BUS_CLK'/>
    <param-binding name='FEATURE_FDCAN' value='false'/>
  </reg-instance>

</vmap>
```

4.3 Spec Publishing Manifest (.pub)

The .pub file governs how register information is processed and merged during PDF and WebHelp publishing. It defines instance selectors — references to a content instance file or a named instance — and hierarchical resolve groups that control how multiple register instances are merged or deduplicated in the output. If multiple instances share the same register structure, the resolve group ensures common register information appears only once, with per-instance address information consolidated in a generated top-level memory map table.

At subsystem and SoC levels, the top-level .pub references IP-level and subsystem-level .pub files, building a hierarchical manifest that resolves the complete publication tree.

```
<!-- ip.pub - spec publishing manifest for FlexCAN IP spec -->
<?xml version='1.0' encoding='UTF-8'?>
<pub id='flexcan-proc' spec='IP_FlexCAN_SPEC'>
```

```

<!-- Register description file references (by ID) -->
<regs-ref id='flexcan-regs' href='../regs/flexcan.regs'/>

<!-- Resolve group 1: publish with FLEXCAN_0 as primary instance -->
<resolve-group id='flexcan-primary' regs-ref='flexcan-regs'>
  <instance-selector vmap-ref='../inst/ip_params_top.vmap'
    instance-id='FLEXCAN_0'/>
  <!-- Address table will list FLEXCAN_0 and FLEXCAN_1 offsets -->
  <address-instance instance-id='FLEXCAN_0' base='0x4A510000'/>
  <address-instance instance-id='FLEXCAN_1' base='0x4A520000'/>
</resolve-group>

</pub>

<!-- top.pub – SoC-level manifest referencing all IP manifests -->
<pub id='imx95-top-proc' spec='SoC_IMX95_SPEC'>
  <include-proc href='../IP_FlexCAN_SPEC/spec_source/manifest/ip.pub'/>
  <include-proc href='../IP_LPSPi_SPEC/spec_source/manifest/ip.pub'/>
  <include-proc href='../IP_LPUART_SPEC/spec_source/manifest/ip.pub'/>
  <!-- ... additional IP manifest references ... -->
</pub>

```

4.4 Register Description File (.regs)

The .regs file is a proprietary XML format analogous in purpose to IPXACT register maps but adapted for the NXP documentation workflow. It stores complete register definitions for an IP block: register name, offset, reset value, access type, and full bit-field definitions including field name, bit range, access type, reset value, and per-value descriptions. The .regs file supports parameterization — field counts and offset calculations can reference parameters defined in the .prm file and resolved by the publishing pipeline.

```

<!-- flexcan.regs – register description file (excerpt) -->
<?xml version='1.0' encoding='UTF-8'?>
<regs id='flexcan-registers' ip='FlexCAN' version='3.2'>

  <register name='MCR' offset='0x000' reset-value='0xD890000F'
    access='RW' description='Module Configuration Register'>
    <field name='MDIS' bits='31' access='RW' reset='1'
      description='Module Disable – disables the FlexCAN module'/>
    <field name='FRZ' bits='30' access='RW' reset='1'
      description='Freeze Enable'/>
    <field name='RFEN' bits='29' access='RW' reset='0'
      description='Rx FIFO Enable'/>
    <field name='HALT' bits='28' access='RW' reset='1'
      description='Halt FlexCAN'/>
    <field name='MAXMB' bits='6:0' access='RW' reset='0x0F'
      description='Number of Last Message Buffer – max value controlled
        by parameter NUM_MB'>
      <!-- Parameterized field: actual count depends on NUM_MB at resolution time -->
      <param-condition param='NUM_MB' op='gt' value='64' applies='extended-mb-range'/>
    </field>
  </register>

  <!-- Parameterized register block: FD-CAN registers, conditional on FEATURE_FDCAN -->
  <register-block condition='FEATURE_FDCAN=true'>
    <register name='FDCTRL' offset='0xC00' reset-value='0x80004100'
      access='RW' description='CAN FD Control Register'>
      <field name='FDRATE' bits='31' access='RW' reset='1'
        description='Bit Rate Switch Enable'/>
    </register>
  </register-block>

```


</regs>

5. Verific-Based RTL Design Analysis and Parameter Extraction

The RTL design analysis stage is the foundation of accurate parameter extraction. The previous LEC-based approach was replaced with Verific Design Automation's parser and elaboration platform — an industry-grade SystemVerilog (IEEE 1800) and VHDL (IEEE 1076) parser that performs full analysis and static RTL elaboration of the design hierarchy.

5.1 Verific Analysis and Elaboration

Verific operates in two stages. Analysis creates parse trees and resolves identifier meanings through type inferencing. RTL elaboration then binds instances to modules, resolves library references, propagates defparam assignments, unrolls generate statements, and checks hierarchical names — producing an elaborated netlist database with complete parameter resolution at every level of hierarchy.

The key advantage for parameter extraction: after RTL elaboration, every parameter override at every level of the design hierarchy is fully resolved. A parameter defined at the chip top level that propagates through a subsystem to an IP instance is captured with its final, fully-resolved value — precisely what documentation needs.

5.2 Flow Inputs

Input	Description
RTL File List	SystemVerilog and/or VHDL source files comprising the SoC or subsystem design. Passed to Verific for analysis and elaboration.
Top-Level Module	The SoC or subsystem top-level module name from which hierarchy walk begins.
Working Directory	Directory for Verific compilation artifacts and output JSON files.
Top Spec Memory Map (XLSX)	Excel file describing each IP instance in the target spec: spec repository path, register description file link (regs-ref), start address, end address, slot size, and address space identifier.

5.3 module_parameters.json Generation

A Python script using Verific's API walks the elaborated hierarchy and dumps a module_parameters.json file. For each IP module instance, the file records the instance's full hierarchical path, module name, resolved parameter values (after all override propagation), and complete register definitions extracted from the elaborated netlist.

```
# verific_extractor.py – walk elaborated hierarchy via Verific Python API
import verific_db as vdb
import json

def extract_module_parameters(top_module: str, filelist: str,
                             work_dir: str) -> dict:
    # Initialize Verific and compile RTL
    vdb.analyze_files(filelist=filelist, work_dir=work_dir)
    vdb.elaborate(top=top_module)

    result = {}
    design = vdb.get_design(top_module)

    for inst in design.iter_instances(recursive=True):
        module = inst.get_module()
        params = {}
        for param in module.get_parameters():
            resolved_val = inst.get_param_value(param.name) # post-elaboration value
            params[param.name] = {
                'value': resolved_val,
                'type': param.data_type,
                'default': param.default_value,
                'description': getattr(param, 'description', '')
            }
```

```

    }

    result[inst.full_path] = {
        'module_name': module.name,
        'instance_name': inst.name,
        'parameters': params,
        'registers': extract_register_defs(inst) # from elaborated netlist
    }
return result

def main():
    data = extract_module_parameters(
        top_module='imx95_top',
        filelist='/work/rtl/imx95.f',
        work_dir='/work/verific_out'
    )
    with open('/work/verific_out/module_parameters.json', 'w') as f:
        json.dump(data, f, indent=2)
    print(f'Extracted {len(data)} instances')

```

5.4 Top Spec Memory Map Processing

In parallel with Verific analysis, the platform reads the Top Spec Memory Map XLSX file to determine which IP specs are required, populate them from their repository paths, and establish the address layout. This file is the single authoritative source for what is in scope for the current SoC or subsystem spec.

```

# memmap_reader.py - read top spec memory map XLSX
import openpyxl

def read_top_memmap(xlsx_path: str) -> list:
    wb = openpyxl.load_workbook(xlsx_path)
    ws = wb.active
    entries = []
    for row in ws.iter_rows(min_row=2, values_only=True):
        ip_name, spec_repo_path, regs_ref, start_addr, end_addr, slot_size, addr_space
= row
        if ip_name: # skip blank rows
            entries.append({
                'ip_name': ip_name,
                'spec_repo_path': spec_repo_path, # e.g. git://repo/IP_FlexCAN_SPEC
                'regs_ref': regs_ref, # e.g. regs/flexcan.regs
                'start_address': int(start_addr, 16) if isinstance(start_addr, str)
else start_addr,
                'end_address': int(end_addr, 16) if isinstance(end_addr, str) else
end_addr,
                'slot_size': slot_size,
                'address_space': addr_space
            })
    return entries

def populate_specs_from_repo(entries: list, work_dir: str):
    for e in entries:
        # Clone or update each IP spec from its repository path
        dest = os.path.join(work_dir, 'specs', e['ip_name'])
        if not os.path.exists(dest):
            subprocess.run(['git', 'clone', e['spec_repo_path'], dest], check=True)
        else:
            subprocess.run(['git', '-C', dest, 'pull'], check=True)

```

6. Non-RTL Parameter Database and Enrichment

Not all parameters governing SoC documentation come from the RTL database. A significant category of parameters — IP revision identifiers, documentation feature flags, chip-tier identifiers, supported protocol variant labels — exist only in the documentation layer. These are not synthesized into RTL and therefore cannot be extracted by Verific.

6.1 Database Design

The non-RTL parameter database is a hierarchical JSON file, not a relational database. This design choice enables it to be version-controlled alongside documentation source, diffed in code review, and edited without database tooling. The file records parameter-to-instance associations across the chip hierarchy, using chip prefixes to select applicable values per target.

```
// non_rtl_params_db.json — hierarchical JSON parameter database (excerpt)
{
  'schema_version': '2.1',
  'parameters': {
    'FEATURE_FDCAN': {
      'type': 'boolean',
      'description': 'FD-CAN capability enabled for this chip configuration',
      'ip': 'FlexCAN',
      'instances': {
        'IMX95': {
          'FLEXCAN_0': true,
          'FLEXCAN_1': false
        },
        'IMX93': {
          'FLEXCAN_0': false,
          'FLEXCAN_1': false
        }
      }
    },
    'IP_REVISION': {
      'type': 'string',
      'description': 'IP design revision for this tape-out',
      'ip': 'FlexCAN',
      'instances': {
        'IMX95': { '_all': '3.2' },
        'IMX93': { '_all': '3.0' }
      }
    }
  },
  'subsystem_mappings': {
    'AUDIO_SS': {
      'IMX95': ['SAI_0', 'SAI_1', 'ASRC_0']
    }
  }
}
```

6.2 RTL + Non-RTL Parameter Merge

After Verific produces `module_parameters.json` and the memory map reader populates the spec directories, the merge step queries the non-RTL parameter database using the chip prefix (e.g., 'IMX95') and instance identifiers to combine both parameter sources into the unified `ip_params_top.vmap` file.

```
# param_merger.py — merge RTL params with non-RTL database
def build_top_vmap(module_params_json: str, memmap_entries: list,
                   non_rtl_db_path: str, chip_prefix: str,
                   output_path: str):
    with open(module_params_json) as f:
        rtl_data = json.load(f)
    with open(non_rtl_db_path) as f:
```

```

non_rtl_db = json.load(f)

root = ET.Element('vmap', chip=chip_prefix)

for entry in memmap_entries:
    ip_name = entry['ip_name']
    inst_path = find_instance_path(rtl_data, ip_name, entry['start_address'])
    rtl_params = rtl_data.get(inst_path, {}).get('parameters', {})

    inst_el = ET.SubElement(root, 'reg-instance',
        id=derive_instance_id(ip_name, entry['start_address']),
        **{'regs-ref': entry['regs_ref'],
          'base-address': hex(entry['start_address'])})

    # Bind RTL parameters
    for pname, pdata in rtl_params.items():
        ET.SubElement(inst_el, 'param-binding', name=pname,
            source='rtl', value=str(pdata['value']))

    # Enrich with non-RTL parameters for this IP and chip
    non_rtl = query_non_rtl(non_rtl_db, ip_name, chip_prefix,
        derive_instance_id(ip_name, entry['start_address']))
    for pname, pval in non_rtl.items():
        ET.SubElement(inst_el, 'param-binding', name=pname,
            source='non-rtl', value=str(pval))

write_xml(root, output_path)

```

7. Automated Top-Level Spec Source Generation

Once the `ip_params_top.vmap` file is built, the pipeline auto-generates the remaining top-level spec source artifacts: the top-level DITA map, the top-level `.pub` publishing manifest, chip-specific DITA topic sections for each IP, and a consolidated SoC-level `.prm` file.

7.1 Top-Level DITA Map Generation

```
# ditamap_gen.py – generate top-level DITA map from vmap + memmap
def generate_top_ditamap(vmap_path: str, memmap_entries: list,
                        soc_chapters_dir: str, output_path: str):
    tree = ET.parse(vmap_path)
    root_el = tree.getroot()

    map_root = ET.Element('map', id=f'{chip_prefix}-reference-manual')
    ET.SubElement(map_root, 'title').text = f'{chip_prefix} Reference Manual'

    # Insert AI-authored SoC chapters first
    for chapter in sorted(glob.glob(soc_chapters_dir + '/*.dita')):
        ET.SubElement(map_root, 'topicref', href=chapter)

    # Insert IP spec ditamaps from vmap reg-instances
    seen_ips = set()
    for reg_inst in root_el.findall('reg-instance'):
        ip_spec_dir = find_ip_spec_dir(reg_inst, memmap_entries)
        ip_ditamap = os.path.join(ip_spec_dir, 'spec_source/maps/ip.ditamap')
        if ip_spec_dir not in seen_ips:
            ref = ET.SubElement(map_root, 'topicref',
                               format='ditamap', href=ip_ditamap)
            ET.SubElement(ET.SubElement(ref, 'topicmeta'),
                          'navtitle').text = derive_nav_title(reg_inst)
            seen_ips.add(ip_spec_dir)

    write_xml(map_root, output_path)
```

7.2 Top-Level Publishing Manifest (.pub) Generation

The top-level `.pub` is generated by iterating over the IP spec directories referenced in the `vmap` file and creating include-proc references to each IP spec's own `.pub` file. The hierarchical structure means that the top-level manifest resolves the entire publication tree.

7.3 Chip-Specific DITA Topic Sections

For each IP instance in the SoC, the pipeline generates chip-specific DITA topic files — typically a chip introduction topic and a chip-specific configuration topic — and inserts references to these into the top-level DITA map and the `vmap` file. This ensures IP-level reuse while accommodating chip-specific narrative content.

8. AI-Powered SoC Chapter Authoring

SoC-level architectural chapters — introductions, clocking overviews, power domain descriptions, interrupt matrix tables, boot sequence narratives — follow predictable structural patterns but are architecturally unique per chip. Rather than requiring technical writers to author these from scratch, the platform uses a Claude-based agent to extract relevant content from the SoC architecture specification and produce DITA-conformant chapter topics automatically.

8.1 Agent Architecture

Input / Output	Description
Input 1	SoC architecture specification document (PDF or extracted text), windowed to the chapter-relevant section
Input 2	Chapter type and structural template defining required DITA elements, topic type, and section hierarchy
Input 3	Merged parameter context (ip_params_top.vmap) providing accurate instance names, addresses, and feature flags for cross-reference accuracy
Output	DITA-conformant XML topic — <concept> for narrative overviews, <reference> for tabular register/address data

8.2 Implementation

```
# soc_chapter_agent.py – Claude-powered SoC chapter generation
import anthropic, json, os

client = anthropic.Anthropic()

SYSTEM_PROMPT = '''You are a semiconductor documentation engineer at NXP Semiconductors.
Generate DITA 1.3 conformant XML chapter topics for SoC Reference Manuals.
Follow these rules:
- Use <concept> topic type for architectural overviews and narratives
- Use <reference> topic type for tabular data (registers, addresses, interrupts)
- Use only values and names present in the provided architecture spec and parameter context
- Never invent or estimate register values, addresses, or instance counts
- Format addresses as 0x-prefixed hex; use consistent decimal for counts
- Output only valid DITA XML. No prose wrapper, no markdown, no code fences.'''

def generate_chapter(arch_spec_section: str, chapter_type: str,
                    params_context: dict, template: str) -> str:
    user_msg = f'''
Chapter type: {chapter_type}
DITA structure template:
{template}

SoC parameter context (use these values for addresses, instance names, counts):
{json.dumps(params_context, indent=2)[:3000]}

Architecture specification section:
{arch_spec_section[:6000]}

Generate the complete DITA topic now.
'''
    response = client.messages.create(
        model='claude-sonnet-4-20250514',
        max_tokens=4096,
```

```
    system=SYSTEM_PROMPT,  
    messages=[{'role': 'user', 'content': user_msg}]  
)  
return response.content[0].text
```


9. DITA-OT Parameter Resolution and Publishing

The publishing pipeline takes the complete spec source tree — DITA maps, topics, .vmap files, .prm files, and .regs files — and produces fully resolved, parameter-free DITA output, followed by standard DITA-OT transformation to PDF or WebHelp.

9.1 Custom Applicability Attributes

All DITA topics, map references, and register files support a custom applicability attribute. This attribute can reference any parameter defined in the active .prm file and express a condition using a value comparison — for example, marking a block of content as applicable only when `FEATURE_FDCAN=true`. During parameter resolution, elements with non-matching applicability conditions are filtered from the resolved DITA output.

```
<!-- Example: parameterized DITA content with applicability attributes -->
<section id='fdcan-operation'
  applies-when='FEATURE_FDCAN=true'>
  <title>CAN FD Operation</title>
  <p>When CAN FD is enabled (FEATURE_FDCAN=true), the FlexCAN module
    supports bit rates up to 8 Mbit/s for the data phase.</p>
</section>

<section id='classic-can-only'
  applies-when='FEATURE_FDCAN=false'>
  <title>Classic CAN Operation</title>
  <p>This instance supports Classical CAN 2.0B only.</p>
</section>

<!-- Parameterized cross-reference using parameter substitution -->
<p>The base address of this instance is
  <ph keyref='BASE_ADDRESS' /> in the SoC memory map.</p>
<p>This instance provides <ph keyref='NUM_MB' /> message buffers.</p>
```

9.2 DITA-OT Wrapper and Resolution Logic

```
# dita_ot_wrapper.py – parameter resolution preprocessing + DITA-OT invocation
import xml.etree.ElementTree as ET
import subprocess, os, glob, shutil

def resolve_and_publish(top_ditamap: str, top_pub: str,
                       output_dir: str, output_format: str = 'pdf'):
    # Step 1: Load all vmap bindings from pub hierarchy
    bindings = load_all_bindings(top_pub) # dict: instance_id -> params

    # Step 2: Resolve all DITA files in the map tree
    resolved_dir = os.path.join(output_dir, 'resolved')
    os.makedirs(resolved_dir, exist_ok=True)

    for dita_file in walk_dita_map(top_ditamap):
        resolved_xml = resolve_dita_file(dita_file, bindings)
        rel_path = os.path.relpath(dita_file,
                                   os.path.dirname(top_ditamap))
        out_path = os.path.join(resolved_dir, rel_path)
        os.makedirs(os.path.dirname(out_path), exist_ok=True)
        with open(out_path, 'w', encoding='utf-8') as f:
            f.write(resolved_xml)

    # Step 3: Convert .regs files to DITA topics
    convert_regs_to_dita(bindings, resolved_dir)

    # Step 4: Run DITA-OT on resolved source
```

```
resolved_map = os.path.join(resolved_dir, os.path.basename(top_ditamap))
subprocess.run([
    'dita', '-i', resolved_map,
    '-f', 'pdf2' if output_format == 'pdf' else 'webhelp-responsive',
    '-o', os.path.join(output_dir, output_format),
    '--args.fo.userconfig=cfg/custom_fo_config.xml'
], check=True)
```

```
def convert_regs_to_dita(bindings: dict, resolved_dir: str):
    # Transform each .regs file into a DITA reference topic
    # with register bit-field diagrams (SVG) and structured tables
    for regs_file in glob.glob(resolved_dir + '/*/*/*.regs', recursive=True):
        inst_bindings = find_bindings_for_regs(bindings, regs_file)
        dita_out = regs_file.replace('.regs', '_regs.dita')
        render_regs_as_dita(regs_file, inst_bindings, dita_out)
```

10. Register Specification Validation

The Verific elaboration output includes not only parameter values but also complete register definitions extracted from the RTL. These are cross-validated against the corresponding .regs files maintained by IP documentation teams. Discrepancies are surfaced as structured validation reports within the VSCode plugin.

10.1 What Is Validated

Validation Check	Description
Register existence	Registers present in RTL but missing from spec, or in spec but absent from RTL
Reset values	RTL reset value vs. spec-documented reset value — mismatches are frequent sources of errata
Field bit ranges	Bit position and width of each named field — mismatches indicate documentation lag behind design changes
Field access types	Read/write access classification (RW, RO, WO, W1C) — differences indicate semantic errors in documentation
Field names	RTL field names vs. spec field names — identifies undocumented fields and naming divergence

```
# reg_validator.py – cross-validate RTL register data vs .regs spec files
def validate_registers(rtl_regs: dict, spec_regs_dir: str) -> list:
    issues = []
    for ip_name, rtl_ip_regs in rtl_regs.items():
        spec_file = find_regs_file(spec_regs_dir, ip_name)
        if not spec_file:
            issues.append({'ip': ip_name, 'severity': 'ERROR',
                          'type': 'missing_spec_file',
                          'message': f'No .regs file found for {ip_name}'})
            continue

        spec_regs = parse_regs_file(spec_file)
        for reg_name, rtl_reg in rtl_ip_regs.items():
            if reg_name not in spec_regs:
                issues.append({'ip': ip_name, 'register': reg_name,
                              'severity': 'ERROR', 'type': 'missing_in_spec'})
                continue

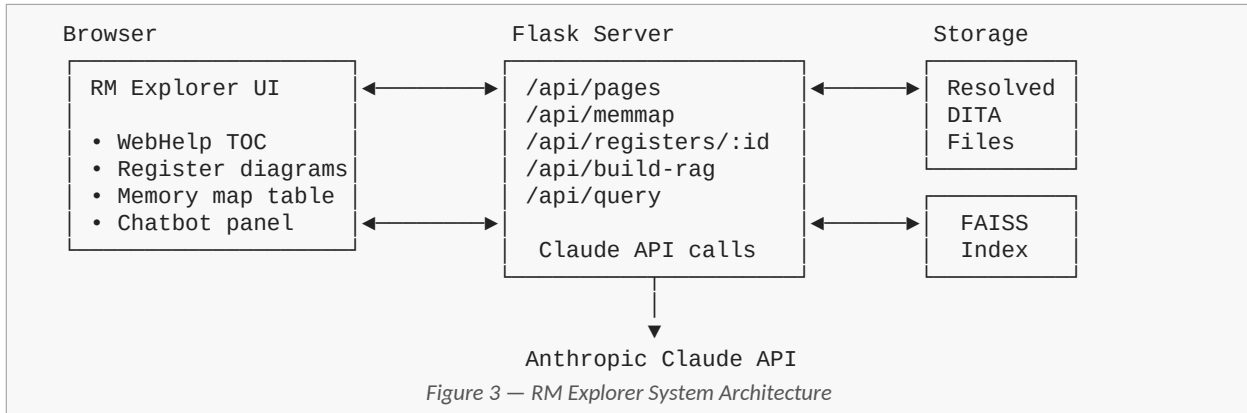
            spec_reg = spec_regs[reg_name]
            if rtl_reg['reset_value'] != spec_reg['reset_value']:
                issues.append({'ip': ip_name, 'register': reg_name,
                              'severity': 'ERROR', 'type': 'reset_mismatch',
                              'rtl_value': rtl_reg['reset_value'],
                              'spec_value': spec_reg['reset_value']})
            for field_name, rtl_field in rtl_reg['fields'].items():
                spec_field = spec_reg['fields'].get(field_name)
                if spec_field and rtl_field['bits'] != spec_field['bits']:
                    issues.append({'ip': ip_name, 'register': reg_name,
                                  'field': field_name,
                                  'severity': 'ERROR', 'type': 'bit_range_mismatch',
                                  'rtl': rtl_field['bits'],
                                  'spec': spec_field['bits']})

    return issues
```

11. RM Explorer — WebHelp Renderer and RAG Chatbot

The RM Explorer is a browser-based application served by a Flask backend. It renders the resolved DITA output as navigable, bookmark-rich WebHelp with a modern UI. Register information (from .regs files transformed during the publishing pipeline) is displayed inline as bit-field diagrams and structured tables. Memory maps for individual IPs and for the full SoC are accessible as sortable, filterable tables.

11.1 System Components



11.2 RAG Engine

```
# rag_engine.py – build RAG knowledge base from resolved DITA
from sentence_transformers import SentenceTransformer
import numpy as np, faiss, glob
import xml.etree.ElementTree as ET

class RagEngine:
    def __init__(self):
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
        self.index = None
        self.chunks = []
        self.chunk_count = 0

    def build_from_dita(self, resolved_dita_dir: str):
        self.chunks = []
        for dita_file in glob.glob(resolved_dita_dir + '/*/*.dita',
                                   recursive=True):
            for chunk in extract_semantic_chunks(dita_file):
                self.chunks.append({'text': chunk['text'],
                                   'source': dita_file,
                                   'section': chunk['section_title']})
        texts = [c['text'] for c in self.chunks]
        embeddings = self.model.encode(texts, show_progress_bar=True)
        self.index = faiss.IndexFlatL2(embeddings.shape[1])
        self.index.add(np.array(embeddings, dtype='float32'))
        self.chunk_count = len(self.chunks)

    def retrieve(self, query: str, top_k: int = 5) -> list:
        q_emb = self.model.encode([query]).astype('float32')
        distances, indices = self.index.search(q_emb, top_k)
        return [self.chunks[i] for i in indices[0]]

    def extract_semantic_chunks(dita_file: str) -> list:
        # Parse DITA and extract section-level text chunks with titles
        tree = ET.parse(dita_file)
        chunks = []
```

```

for section in tree.findall('.//{*}section') + tree.findall('.//{*}refsyn'):
    title_el = section.find('{*}title')
    title = title_el.text if title_el is not None else 'Untitled'
    text = ' '.join(section.itertext()).strip()
    if len(text) > 100: # skip trivially short chunks
        chunks.append({'text': text, 'section_title': title})
return chunks

```

11.3 Claude-Powered Natural Language Chatbot

```

# claude_chatbot.py – natural language RM query interface
import anthropic

client = anthropic.Anthropic()

SYSTEM = '''You are a hardware documentation assistant for NXP semiconductor products.
Use only the provided context from the Reference Manual to answer questions.
If a register value, address, or parameter is asked about, quote the exact value from
context.
If the information is not in the provided context, say: 'This information is not in the
loaded RM section. Try searching a different section.'
Always cite the section name from which you retrieved each piece of information.'''

def answer_query(query: str, rag_engine, conversation_history: list) -> dict:
    context_chunks = rag_engine.retrieve(query, top_k=5)
    context_text = ' '.join([
        f'[Section: {c["section"]} {c["text"]} '
        for c in context_chunks
    ])
    # Append to conversation for multi-turn coherence
    conversation_history.append({
        'role': 'user',
        'content': f'Context from RM: {context_text} Question: {query}'
    })
    response = client.messages.create(
        model='claude-sonnet-4-20250514',
        max_tokens=1024,
        system=SYSTEM,
        messages=conversation_history
    )
    answer = response.content[0].text
    # Store assistant response for next turn
    conversation_history.append({'role': 'assistant', 'content': answer})
    return {'answer': answer, 'sources': [c['section'] for c in context_chunks]}

```

12. Technology Stack

Component	Technology	Notes
VSCoDe Plugin	TypeScript	VSCoDe Extension API, JSON-RPC backend bridge, sidebar panel, command palette integration
RTL Analysis	Python + Verific API	Verific SystemVerilog/VHDL parser and elaborator; Python API for hierarchy walk and parameter dump
Memory Map Processing	Python, openpyxl	XLSX memory map reading, spec repository population via git
Non-RTL Param DB	JSON	Hierarchical JSON file; version-controlled alongside documentation source
Parameter Merge	Python, ElementTree	RTL + non-RTL merge into unified .vmap XML
Spec Source Generation	Python, Jinja2, ElementTree	Top DITA map, .pub, chip sections, .prm generation
AI Chapter Authoring	Python, Anthropic SDK	Claude API (claude-sonnet-4), DITA-conformant XML output
DITA-OT Publishing	Python, Java, DITA-OT 3.x	Custom DITA-OT wrapper; XSLT transforms for .regs→DITA; PDF2 and WebHelp-Responsive formats
Register Validation	Python, ElementTree	RTL-vs-spec cross-validation with structured issue reporting
RM Explorer Backend	Python, Flask	REST API serving resolved DITA content, register data, memory maps
RAG Engine	Python, FAISS, sentence-transformers	Semantic chunking of resolved DITA; FAISS L2 index; all-MiniLM-L6-v2 embeddings
Chatbot	Python, Anthropic SDK	Multi-turn conversation with RAG context; claude-sonnet-4 via Anthropic Python SDK
Source Format	DITA 1.3, XML, JSON	Oxygen XML authoring environment; custom applicability attributes; .prm/.vmap/.pub schemas